

DVSA Vulnerability Discovery and Remediation

ICS-344 Information Security Course Project

Team 17

King Fahd University of Petroleum and Minerals

Term 252

Project Scope

- Controlled university lab project using OWASP DVSA in a non-production AWS account.
- Required coverage: all 10 official vulnerability lessons.
- Deliverables: written report, presentation slides, repository evidence, fixes, and demo video.
- All sensitive values are redacted: JWTs, AWS account IDs, credentials, emails, and private identifiers.

Serverless Architecture

- Frontend: S3 static website.
- Entry point: API Gateway routes browser and direct API requests.
- Compute: Lambda functions implement order, billing, receipt, and admin workflows.
- Data and services: DynamoDB, S3 receipts, SQS, SES, CloudWatch, CloudTrail, and IAM.
- Main security controls: input validation, JWT verification, authorization, IAM least privilege, throttling, and safe errors.

- ① Establish the intended normal workflow.
- ② Capture the request, AWS resource, function, or dependency involved.
- ③ Reproduce the vulnerable behavior in the DVSA lab only.
- ④ Store proof as screenshots, logs, API output, policy simulator results, or report sources.
- ⑤ Document the fix and repeat the test after remediation.
- ⑥ Confirm normal behavior still works.

Lesson Coverage Map

No.	Lesson	Primary Evidence Location
1	Event Injection	vulnerabilities/lesson-01, report-source.pdf
2	Broken Authentication	screenshots/lesson-02/from-report
3	Sensitive Data Exposure	vulnerabilities/lesson-03, report-source.pdf
4	Insecure Cloud Configuration	screenshots/lesson-04/from-report
5	Broken Access Control	screenshots/lesson-05/from-report
6	Denial of Service	screenshots/lesson-06
7	Over-Privileged Function	screenshots/lesson-07
8	Logic Vulnerability	screenshots/lesson-08/from-report
9	Vulnerable Dependencies	vulnerabilities/lesson-09, report-source.pdf
10	Unhandled Exceptions	screenshots/lesson-10/from-report

Lesson 1: Event Injection

Affected Component

API Gateway request path and order-processing Lambda.

Root Cause and Impact

Unsafe deserialization or dynamic input handling can treat request data as executable behavior inside Lambda. Impact: backend code execution in the lab function context.

Evidence

Redacted payload in `redacted-requests.txt`; Lesson 1/9 report-source PDF; CloudWatch proof is the key evidence source when API Gateway returns a generic error.

Fix and Verification

Remove unsafe deserialization, use strict JSON parsing and schema validation, then repeat the payload and confirm it is rejected without backend execution.

Lesson 2: Broken Authentication

Affected Component

JWT-based order API identity handling.

Root Cause and Impact

The backend trusts decoded JWT claims without proving token integrity. Impact: forged claims can impersonate another user and access order data.

Evidence

Imported DOCX report plus 8 extracted screenshots under `screenshots/lesson-02/from-report`.

Fix and Verification

Verify JWT signature, issuer, audience, expiry, and subject. Re-test forged token: it must fail while valid user tokens still work.

Lesson 3: Sensitive Data Exposure

Affected Component

Receipt/data exposure path involving API output, S3, and backend receipt handling.

Root Cause and Impact

Receipt data or links are exposed without sufficient ownership checks. Impact: private receipt or account-related data can be visible outside the intended boundary.

Evidence

Lesson 3 report-source PDF and sanitized response sample in `redacted-requests.txt`.

Fix and Verification

Add ownership checks, keep S3 objects private, and generate short-lived links only after authorization. Verify unauthorized access fails.

Lesson 4: Insecure Cloud Configuration

Affected Component

S3 receipt bucket and receipt-processing Lambda trigger.

Root Cause and Impact

Bucket access or object-processing trust is too permissive. Impact: untrusted objects can enter a trusted backend workflow.

Evidence

Imported DOCX report plus 12 extracted screenshots under `screenshots/lesson-04/from-report`.

Fix and Verification

Enable Block Public Access, restrict bucket policy/ACLs, validate object keys and sources, then verify unauthorized upload/access fails.

Lesson 5: Broken Access Control

Affected Component

Administrative order update workflow and Lambda invocation permissions.

Root Cause and Impact

A valid normal user context can reach privileged order-update behavior without required role checks.
Impact: unauthorized order state modification.

Evidence

Imported DOCX report plus 3 extracted screenshots under `screenshots/lesson-05/from-report`.

Fix and Verification

Enforce admin group/role checks and narrow Lambda invoke permissions. Verify normal-user admin update attempts are denied.

Lesson 6: Denial of Service

Affected Component

Billing action on `/dvsa/order` and `DVSA-ORDER-BILLING`.

Root Cause and Impact

Duplicate or concurrent billing bursts are not controlled enough. Impact: availability degradation and backend HTTP 500/502 failures during controlled parallel testing.

Evidence

8 screenshots under `screenshots/lesson-06`, including PowerShell results, CloudWatch logs, Lambda metrics, and API Gateway throttling.

Fix and Verification

Apply API Gateway throttling and backend idempotency/rate controls. Verify excess requests degrade safely and normal checkout works.

Lesson 7: Over-Privileged Function

Affected Component

DVSA-SEND-RECEIPT-EMAIL Lambda execution role.

Root Cause and Impact

The role grants broader permissions than the receipt workflow requires. Impact: increased blast radius if the function is abused.

Evidence

10 screenshots under `screenshots/lesson-07`, including Lambda role, broad SES evidence, IAM Policy Simulator, and post-fix workflow.

Fix and Verification

Replace broad policies with least-privilege IAM. Verify unrelated actions are denied and normal order/receipt behavior still works.

Lesson 8: Logic Vulnerability

Affected Component

Order billing/update state machine and DynamoDB order records.

Root Cause and Impact

Billing and update operations can race because state transitions are not atomic. Impact: final order state can differ from what was billed.

Evidence

Imported DOCX report plus 13 extracted screenshots under `screenshots/lesson-08/from-report`.

Fix and Verification

Use DynamoDB conditional writes or transactions and lock updates once billing begins. Verify conflicting operations are rejected.

Lesson 9: Vulnerable Dependencies

Affected Component

Node.js Lambda package/dependency path.

Root Cause and Impact

An unsafe dependency enables dangerous deserialization behavior. Impact: dependency behavior supports event injection risk.

Evidence

Lesson 1/9 report-source PDF and sanitized dependency note in `redacted-requests.txt`.

Fix and Verification

Remove or replace risky packages, pin reviewed versions, and scan dependencies. Verify injection no longer executes and scan is clean.

Lesson 10: Unhandled Exceptions

Affected Component

Malformed input handling in the order API/Lambda path.

Root Cause and Impact

Missing validation and handler-level exception control leak implementation details. Impact: file paths, exception names, or code references may appear in responses.

Evidence

Imported DOCX report plus 2 extracted screenshots under `screenshots/lesson-10/from-report`.

Fix and Verification

Validate required fields and centralize safe error handling. Verify malformed requests return generic client-safe errors only.

Repository Completeness

- Every lesson folder has `README.md`, `notes.md`, `redacted-requests.txt`, and evidence subfolders.
- Every fix folder has before/after artifacts and explanation files.
- Lessons 1, 3, and 9 include PDF report sources.
- Lessons 2, 4, 5, 8, and 10 include DOCX report sources and extracted screenshots.
- Lessons 6 and 7 include direct screenshot evidence from the controlled tests and AWS Console.

Remediation Themes Across All Lessons

- Strict input validation and safe parsing.
- Verified authentication and server-side authorization.
- Least-privilege IAM for Lambda execution roles.
- Secure S3 configuration and private receipt access.
- Atomic workflow updates for state-sensitive operations.
- API throttling, idempotency, and safe error handling.

Final Takeaways

- DVSA shows that serverless risk often appears between AWS services, IAM, and workflow logic.
- Good evidence connects intended behavior, exploit behavior, fix, and post-fix verification.
- The same controls repeat across lessons: validate, verify, authorize, limit permissions, throttle, and log safely.
- Repository artifacts must stay redacted and organized so the grader can follow each lesson quickly.